

Lab 12: Thread Synchronization and Mutexes

In this lab, we will study thread synchronization and mutexes. A mutual exclusion (mutex) is a program object that prevents simultaneous access to a shared resource. This concept is used in concurrent programming with a critical section, a piece of code in which processes or threads access a shared resource. Only one thread owns the mutex at a time. Thus, a mutex with a unique name is created when a program starts. When a thread holds a resource, it has to lock the mutex from other threads to prevent concurrent access to the resource. Upon releasing the resource, the thread unlocks the mutex. **Mutex functions provide for creating, destroying, locking, and unlocking mutexes. These are supplemented by mutex attribute functions that set or modify attributes associated with mutexes.**

13.1 Synchronization

Thread synchronization is defined as a mechanism that ensures that two or more concurrent processes or threads do not simultaneously execute some particular program segment, known as a critical section.

Thread Synchronization Problems

An example code to study synchronization problems:

Example 1

The following code tries to implement a global variable counter from 2 threads. The final value of the counter should reflect all 6 implementations by both threads.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

int max;
int counter = 0; // shared global variable

void *mythread(void *arg) {
    char *letter = arg;
    int i; // stack (private per thread)
    printf("%s: begin [addr of i: %p]\n", letter, &i);
    for (i = 0; i < max; i++) {
        counter = counter + 1; // shared: only one
    }
    printf("%s: done\n", letter);
    return NULL;
}
```

```
int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("usage: main-first <loopcount>\n");
        exit(1);
    }
    max = atoi(argv[1]);
    pthread_t p1, p2;

    printf("main: begin [counter = %d] [%x]\n", counter, (unsigned int)
&counter);

    pthread_create(&p1, NULL, mythread, "A");
    pthread_create(&p2, NULL, mythread, "B");

    // join waits for the threads to finish
    pthread_join(p1, NULL);    pthread_join(p2,
NULL);

    printf("main: done\n [counter: %d]\n [should: %d]\n", counter, max * 2);
    return 0;
}
```

mumtaz@hpc:~

```
[mumtaz@hpc ~]$ nano abc.c
[mumtaz@hpc ~]$ gcc abc.c -o as -lpthread
abc.c: In function 'main':
abc.c:31: warning: cast from pointer to integer of different size
[mumtaz@hpc ~]$ ./as 1000
main: begin [counter = 0] [600d00]
A: begin [addr of i: 0x7fe8cea6eea4]
A: done
B: begin [addr of i: 0x7fe8ce06dea4]
B: done
main: done
[counter: 2000]
[should: 2000]
[mumtaz@hpc ~]$ ./as 1000000
main: begin [counter = 0] [600d00]
A: begin [addr of i: 0x7f51b2daeea4]
B: begin [addr of i: 0x7f51b23adea4]
B: done
A: done
main: done
[counter: 1352941]
[should: 2000000]
[mumtaz@hpc ~]$ ./as 1000000000
main: begin [counter = 0] [600d00]
A: begin [addr of i: 0x7fa9ded4eea4]
B: begin [addr of i: 0x7fa9de34dea4]
B: done
A: done
main: done
[counter: 1489559137]
[should: 2000000000]
[mumtaz@hpc ~]$
```

11.2 Race Condition

What we have demonstrated here is called a **race condition** (or, more specifically, a data race): the results depend on the timing execution of the code. With some bad luck (i.e., context switches that occur at untimely points in the execution), we get the wrong result. In fact, we may get a different result each time; thus, instead of a nice deterministic computation (which we are used to from computers), we call this result indeterminate, where it is not known what the output will be and it is indeed likely to be different across runs. A race condition (or data race) arises if multiple threads of execution enter the critical section roughly simultaneously; both attempt to update the shared data structure, leading to a surprising (and perhaps undesirable) outcome.

13.3 Critical Section

Because multiple threads executing this code can result in a race condition, we call this code a critical section. A critical section is a piece of code that accesses a shared variable (or, more generally, a shared resource) and must not be concurrently executed by more than one thread. The critical section is a piece of code that accesses a shared resource, usually a variable or data structure.

Example 2

The following code uses a global mutex to protect the critical section of each thread and prevents them from corrupting each other values.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
int max;
pthread_mutex_t lock;
int counter = 0; // shared global variable
void *mythread(void *arg) {
    char *letter = arg;
    int i; // stack (private per thread)
    printf("%s: begin [addr of i: %p]\n", letter, &i);
    for (i = 0; i < max; i++) {
        pthread_mutex_lock(&lock);
        counter = counter + 1; // shared: only one
        pthread_mutex_unlock(&lock);
    }
    printf("%s: done\n", letter);
    return NULL;
}
int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "usage: main-first <loopcount>\n");
        exit(1);
    }
    max = atoi(argv[1]);
    pthread_t p1, p2;
    printf("main: begin [counter = %d] [%x]\n", counter,
        (unsigned int) &counter);
    pthread_create(&p1, NULL, mythread, "A");
    pthread_create(&p2, NULL, mythread, "B");
    // join waits for the threads to finish
    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    printf("main: done\n [counter: %d]\n [should: %d]\n",
        counter, max*2);
}
```

```
    return 0;
}
```

Output:

```
ubuntu@ubuntu-desktop:~/Downloads$ ./mumtaz
usage: main-first <loopcount>
ubuntu@ubuntu-desktop:~/Downloads$ ./mumtaz 100
main: begin [counter = 0] [58d32088]
A: begin [addr of i: 0x7fd9ef204e3c]
A: done
B: begin [addr of i: 0x7fd9eea03e3c]
B: done
main: done
[counter: 200]
[should: 200]
ubuntu@ubuntu-desktop:~/Downloads$ ./mumtaz 10000
main: begin [counter = 0] [eebfb088]
A: begin [addr of i: 0x7f83e5340e3c]
B: begin [addr of i: 0x7f83e4b3fe3c]
A: done
B: done
main: done
[counter: 20000]
[should: 20000]
ubuntu@ubuntu-desktop:~/Downloads$ 200000
200000: command not found
ubuntu@ubuntu-desktop:~/Downloads$ ./mumtaz 20000
main: begin [counter = 0] [65e03088]
A: begin [addr of i: 0x7fc935330e3c]
B: begin [addr of i: 0x7fc934b2fe3c]
B: done
A: done
main: done
[counter: 40000]
[should: 40000]
```

Example 3

In this example, two threads(jobs) are created, and in the start function of these threads, a counter is maintained to get the logs about the job number which is started and when it is completed.

```
#include <pthread.h>
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
pthread_t tid[2]; int
counter; void*
trythis(void* arg) {
    unsigned long i = 0;
    counter += 1;
    printf("\n Job %d has started\n", counter);
    for (i = 0; i < (0xFFFFFFFF); i++); // Busy-wait loop
    printf("\n Job %d has finished\n", counter);
    return NULL;
}
int main(void) {
    int i = 0;
    int error;
    while (i < 2) {
        error = pthread_create(&(tid[i]), NULL, &trythis, NULL);
        if (error != 0) {
            printf("\nThread can't be created: [%s]\n", strerror(error));
        }
        i++;
    }
}
```

```
pthread_join(tid[0], NULL);

pthread_join(tid[1], NULL);

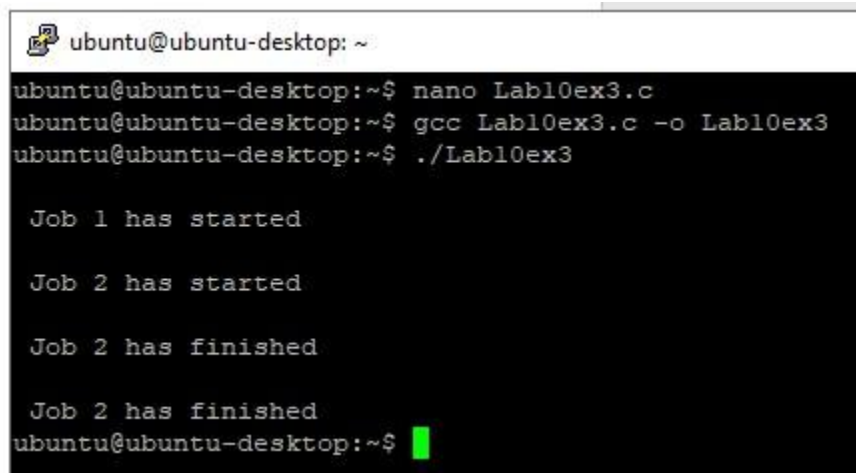
return 0;

}
```

To compile a multithreaded program using gcc, we need to link it with the pthreads library. Following is the command used to compile the program.

```
gfg@ubuntu:~/ $ gcc filename.c -lpthread
```

Output :



```
ubuntu@ubuntu-desktop: ~  
ubuntu@ubuntu-desktop:~$ nano Lab10ex3.c  
ubuntu@ubuntu-desktop:~$ gcc Lab10ex3.c -o Lab10ex3  
ubuntu@ubuntu-desktop:~$ ./Lab10ex3  
  
Job 1 has started  
  
Job 2 has started  
  
Job 2 has finished  
  
Job 2 has finished  
ubuntu@ubuntu-desktop:~$
```

Explanation of the above code

Problem: From the last two logs, one can see that the log '*Job 2 has finished*' is repeated twice while no log for '*Job 1 has finished*' is seen.

Why has it occurred?

On observing closely and visualizing the execution of the code, we can see that :

- The log '*Job 2 has started*' is printed just after '*Job 1 has Started*', so it can easily be concluded that while thread 1 was processing, the scheduler scheduled thread 2.
- If we take the above assumption as true, then the value of the '*counter*' variable got incremented again before job 1 got finished.
- So, when Job 1 actually got finished, then the wrong value of counter produced the log '*Job 2 has finished*' followed by the '*Job 2 has finished*' for the actual job 2 or vice versa as it is dependent on the scheduler.
- So we see that it is not the repetitive log but the wrong value of the '*counter*' variable that is the problem.

- The actual problem was using the variable 'counter' by a second thread when the first thread was using or about to use it.
- In other words, we can say that lack of synchronization between the threads while using the shared resource 'counter' caused the problems, or in a word, we can say that this problem happened due to a 'Synchronization problem' between two threads.

How to solve it?

The most popular way of achieving thread synchronization is by using **Mutexes**.

13.4 Mutex Implementation (Code)

- A Mutex is a lock that we set before using a shared resource and release after using it.
- When the lock is set, no other thread can access the locked region of the code.
- So we see that even if thread 2 is scheduled while thread 1 was not done accessing the shared resource and the code is locked by thread 1 using mutexes, then thread 2 cannot even access that region of code.
- So this ensures synchronized access to shared resources in the code.

Working of a mutex

1. Suppose one thread has locked a region of code using mutex and is executing that piece of code.
2. Now, if the scheduler decides to do a context switch, all the other threads that are ready to execute the same region are unblocked.
3. Only one of all the threads would make it to the execution, but if this thread tries to execute the same region of code that is already locked, then it will again go to sleep.
4. Context switch will take place again and again, but no thread would be able to execute the locked region of code until the mutex lock over it is released.
5. Mutex lock will only be released by the thread that locked it.
6. So this ensures that once a thread has locked a piece of code, then no other thread can execute the same region until it is unlocked by the thread that locked it. Hence, this system ensures synchronization among the threads while working on shared resources.

Example 4

An example to show how mutexes are used for thread synchronization.

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
pthread_t tid[2]; int
counter; pthread_mutex_t
lock; void*
trythis(void* arg)
{
```



```

pthread_mutex_lock(&lock);
unsigned long i = 0;
counter += 1;
printf("\n Job %d has started\n", counter);
for (i = 0; i < (0xFFFFFFFF); i++)
    ;
    printf("\n Job %d has finished\n", counter);
pthread_mutex_unlock(&lock);
return NULL;
}
int main(void)
{
    int i = 0;
    int error;
    if (pthread_mutex_init(&lock, NULL) != 0) {
        printf("\n mutex init has failed\n");
        return 1;
    }
    while (i < 2) { error = pthread_create(&(tid[i]),
        NULL, &trythis, NULL);
        if (error != 0)
            printf("\nThread can't be created :[%s]",
                strerror(error));
            i++;
        }
    pthread_join(tid[0], NULL);
    pthread_join(tid[1], NULL);
    pthread_mutex_destroy(&lock);
    return 0;
}

```

In the above code:

- A mutex is initialized at the beginning of the main function.
- The same mutex is locked in the 'trythis()' function while using the shared resource 'counter'.
- At the end of the function 'trythis()', the same mutex is unlocked.
- At the end of the main function, when both the threads are done, the mutex is destroyed.

Output :

```
ubuntu@ubuntu-desktop: ~  
ubuntu@ubuntu-desktop:~$ nano Lab10ex4.c  
ubuntu@ubuntu-desktop:~$ gcc Lab10ex4.c -o Lab10ex4  
ubuntu@ubuntu-desktop:~$ ./Lab10ex4  
  
Job 1 has started  
  
Job 1 has finished  
  
Job 2 has started  
  
Job 2 has finished  
ubuntu@ubuntu-desktop:~$
```

Example 5

```
#include <stdio.h>  
#include <pthread.h>  
int run(void *arg)  
{  
    (void)arg;    static int serial = 0;    //  
    Shared static variable!    printf("Thread running!  
%d\n", serial);  
    serial++;  
    return 0;  
}  
#define THREAD_COUNT 10  
int main(void)  
{  
    pthread_t t[THREAD_COUNT];  
    //thrd_t t[THREAD_COUNT];    for (int i  
= 0; i < THREAD_COUNT; i++) {  
        pthread_create((t + i), NULL, run, (void *) (t+1));  
        //thrd_create(t + i, run, NULL);  
    }  
    for (int i = 0; i < THREAD_COUNT; i++) {  
pthread_join(t[i], NULL);  
//thrd_join(t[i], NULL);  
}
```

```
}  
}
```

When I run this code, I get something that looks like this:

```
ubuntu@ubuntu-desktop: ~/Downloads  
ubuntu@ubuntu-desktop:~/Downloads$ ./latifl  
Thread running! 0  
Thread running! 1  
Thread running! 0  
Thread running! 0  
Thread running! 4  
Thread running! 0  
Thread running! 6  
Thread running! 6  
Thread running! 6  
Thread running! 8  
ubuntu@ubuntu-desktop:~/Downloads$
```

Explanation of the above program

Clearly, multiple threads are getting in there and running the `printf()` before anyone gets a change to update the `serial` variable.

What we want to do is wrap the getting of the variable and set it into a single mutex-protected stretch of code.

We'll add a new variable to represent the mutex of type `pthread_mutex_t` in the file scope, initialize it, and then the threads can lock and unlock it in the `run()` function.

Example 6

```
#include <stdio.h>  
#include <pthread.h>  
pthread_mutex_t serial_mtx;      // <-- MUTEX VARIABLE  
void * run(void *arg)  
{  
    (void)arg;    static int serial = 0;    //  
    Shared static variable!  
  
    // Acquire the mutex--all threads will block on this call  
    until  
    // they get the lock:  
    pthread_mutex_lock(&serial_mtx);      // <-- ACQUIRE MUTEX  
    printf("Thread running! %d\n", serial);  
    serial++;  
    // Done getting and setting the data, so free the lock.  
    This //will unblock threads on the mutex_lock() call:  
    pthread_mutex_unlock(&serial_mtx);    // <-- RELEASE MUTEX
```

```

        return 0;
    }
#define THREAD_COUNT 10
int main(void)
{
    pthread_t t[THREAD_COUNT];    // Initialize the mutex
    variable, indicating this is a normal
    // no-frills, mutex:
    pthread_mutex_init(&serial_mtx, pthread_mutex_lock); // <--
//CREATE MUTEX
    for (int i = 0; i < THREAD_COUNT; i++) {
        pthread_create((t + i), NULL, run, (void *) (t+1));
    }
    for (int i = 0; i < THREAD_COUNT; i++) {
        pthread_join(t[i], NULL);
        // Done with the mutex, destroy it:
        pthread_mutex_destroy(&serial_mtx); // <-- DESTROY MUTEX
    }
}

```

Explanation of the above program

See how we initialize and destroy the mutex on lines 38 and 50 of the **main()**. But each individual thread acquires the mutex on line 15 and releases it on line 24.

In

between `pthread_mutex_lock()` and `pthread_mutex_unlock()`

the

And now
output!

```

-desktop:~/Downloads$ ./la
g! 0
g! 1
g! 2
g! 3
g! 4
g! 5
g! 6
g! 7
g! 8
g! 9
-desktop:~/Downloads$

```

is
the
critical
section,

Mutex Operat

1. Initialize the mutex.
2. Lock the mutex before entering the critical section.
3. Execute the critical section (access the shared resource).
4. Unlock the mutex to let other threads access the resource.
5. Destroy the mutex when it's no longer needed.

How Deadlock Occurs

For a deadlock to occur, the following four conditions must be present (known as the **Coffman conditions**):

1. **Mutual Exclusion:** At least one resource must be held in a nonshareable mode.
2. **Hold and Wait:** A thread holds one resource while waiting to acquire additional resources.
3. **No Preemption:** Resources cannot be forcibly removed from threads holding them.
4. **Circular Wait:** A circular chain of threads exists where each thread is waiting for a resource held by the next thread.

Code Example of Deadlock

Here, Thread 1 and Thread 2 each lock one mutex (`mutex1` and `mutex2`, respectively) and then attempt to lock the other. Since each is holding a mutex and waiting for the other, a deadlock occurs.

```
#include <pthread.h>
```

```
#include <stdio.h> #include
```

```
<unistd.h>
```

```
pthread_mutex_t mutex1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t mutex2 = PTHREAD_MUTEX_INITIALIZER; void*
```

```
thread1_func(void* arg) { pthread_mutex_lock(&mutex1);
```

```
printf("Thread 1: Locked mutex1\n"); sleep(1);
```

```

    printf("Thread 1: Waiting for mutex2\n");

pthread_mutex_lock(&mutex2); // Deadlock occurs here

pthread_mutex_unlock(&mutex2);

pthread_mutex_unlock(&mutex1); return NULL;

}

void* thread2_func(void* arg) {

pthread_mutex_lock(&mutex2); printf("Thread 2:

Locked mutex2\n"); sleep(1);

    printf("Thread 2: Waiting for mutex1\n"); pthread_mutex_lock(&mutex1); //

Deadlock occurs here pthread_mutex_unlock(&mutex1);

pthread_mutex_unlock(&mutex2); return NULL;

} int main() {

    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, thread1_func, NULL);

pthread_create(&thread2, NULL, thread2_func, NULL);

pthread_join(thread1, NULL); pthread_join(thread2, NULL);

return 0;

}

```

Tasks:

Q1: Write a program with multiple threads incrementing and decrementing a shared resource (e.g., a bank account balance). Use a mutex to ensure that the balance is correctly updated.

Q2: Imagine you are simulating a banking system where two accounts need to transfer money between each other. However, if both accounts try to lock each other simultaneously (to complete the transfer), a deadlock might occur. You'll first simulate this deadlock situation and then resolve it using mutexes to synchronize the access to these accounts.

Q3: Write a program that causes a deadlock by creating two threads, where each thread locks one mutex and waits for the other. Simulate the deadlock condition and observe the program behavior. Modify the program to resolve the deadlock by implementing a solution using a timeout for mutex locking.

Task Submission Guidelines:

- Make a document and paste all tasks solution there (Not screenshot) and save it as pdf.
- Along with each task solution, also upload the screenshot of the output.
- Include your name and roll no. at the front page.
- Please submit your file in this format (Roll_number-name) i.e (24I-1234-Ali.pdf).
- Files other than pdf will not be accepted and will be marked as 0.